

CENTRO UNIVERSITÁRIO — ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Guia de Apresentação e Defesa

Roteiro técnico para a banca — segurança, breakpoints e testes

Integrantes do grupo:

[Preencher: Nome Completo — Matrícula]

[Preencher: Nome Completo — Matrícula]

Backend: Java 21 · Spring Boot · PostgreSQL | Frontend: React · TypeScript · Vite
junho de 2026

Sumário

1. Visão Geral e Roteiro de Defesa
2. Segurança — Detalhamento Técnico
3. Guia de Breakpoints (Demonstração ao Vivo)
4. Testes da API (Postman / curl)
5. Testes Automatizados (JUnit)
6. Arquitetura do Frontend

Arquitetura, Segurança e Guia de Defesa

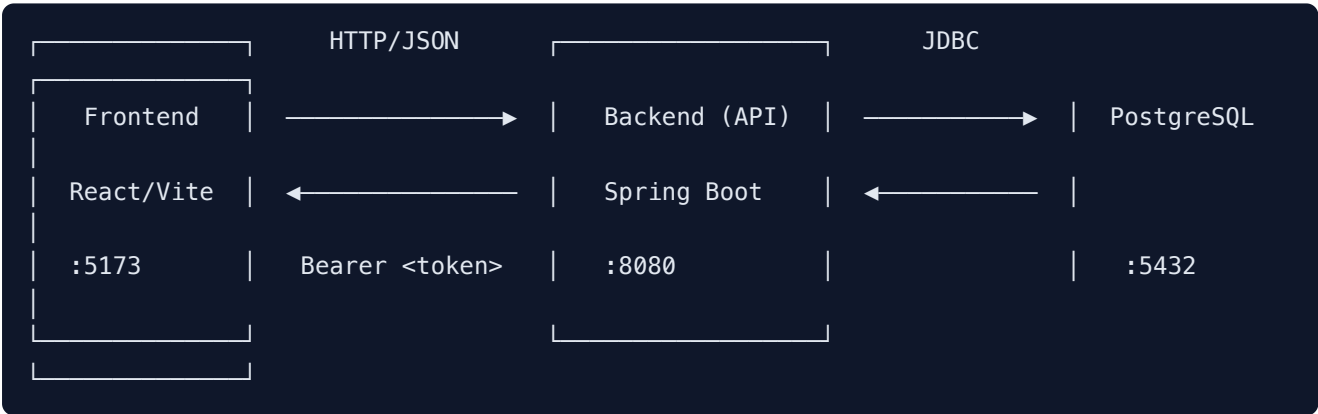
Documento de apoio para o estudo e a apresentação do **Controle Financeiro** à banca. Reúne a visão de arquitetura, o detalhamento da camada de segurança e um roteiro de perguntas e respostas para a defesa ao vivo.

1. Visão geral

O sistema é uma aplicação web de **controle financeiro pessoal**. O usuário cadastra **receitas** e **despesas**, organiza-as em **categorias** e acompanha o resultado em um **dashboard** (saldo, totais do mês, gastos por categoria e fluxo de caixa dos últimos 6 meses). Há ainda um **lançamento automático da renda mensal** no 5º dia útil de cada mês.

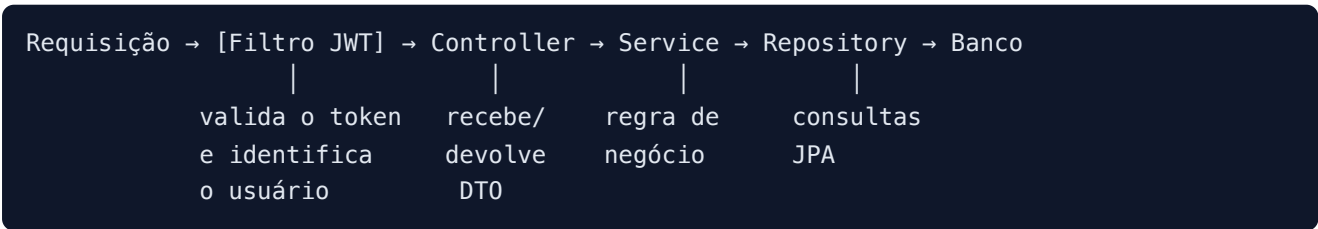
A aplicação é dividida em duas partes independentes:

- **Backend** — API REST em Java/Spring Boot, responsável pela regra de negócio, segurança e persistência.
- **Frontend** — SPA (Single Page Application) em React, que consome a API.



2. Arquitetura do backend (camadas)

O backend segue o padrão clássico em camadas. Cada requisição autenticada percorre o caminho abaixo:



Camada	Pacote	Responsabilidade
Segurança	<code>security/</code>	Valida o token JWT a cada requisição e identifica o usuário.
Controller	<code>controller/</code>	Expõe os endpoints REST; recebe e devolve DTOs (nunca a entidade).
Service	<code>service/</code>	Concentra a regra de negócio; sempre opera no escopo do usuário logado.
Repository	<code>repository/</code>	Interfaces Spring Data JPA — o acesso ao banco.
Entity	<code>entity/</code>	Entidades JPA que viram tabelas.
DTO	<code>dto/</code>	record s de entrada/saída da API.
Job	<code>jobs/</code>	Rotinas agendadas (lançamento da renda, heartbeat).

Por que separar Controller / Service / Repository? Para isolar responsabilidades: o controller cuida do protocolo HTTP, o service da regra de negócio e o repository do banco. Isso facilita teste, manutenção e troca de uma camada sem afetar as outras.

Por que DTO em vez de devolver a entidade? Para não expor dados sensíveis (ex.: o hash da senha do usuário nunca sai da API) e para desacoplar o formato do banco do formato da API.

3. Stack tecnológica e justificativas

Camada	Tecnologia	Por que
Linguagem	Java 21	LTS, tipagem forte, maduro no mercado.
Framework	Spring Boot 3.3	Padrão de mercado para APIs REST; injeção de dependência, segurança e JPA integrados.
Segurança	Spring Security + JWT	Autenticação stateless, sem guardar sessão no servidor.
Persistência	JPA / Hibernate	Mapeia objetos Java para tabelas; gera SQL automaticamente.
Banco	PostgreSQL	Relacional, robusto, open-source.
Boilerplate	Lombok	Gera getters/setters/builders, reduz código repetitivo.
Frontend	React 19 + TypeScript + Vite	SPA reativa, tipada, com build rápido.
UI	PrimeReact + PrimeFlex	Biblioteca de componentes prontos (tabelas, formulários, diálogos).
Estado	Zustand	Gerência de estado simples; guarda o token autenticado.
Gráficos	Chart.js	Pizza (gastos por categoria) e barras (fluxo de caixa).

Camada	Tecnologia	Por que
Formulários	React Hook Form + Zod	Validação declarativa dos formulários.
HTTP	Axios	Cliente HTTP com <i>interceptors</i> para anexar o token.

4. Segurança (ponto central da defesa)

4.1 Autenticação por token JWT

O login **não cria sessão no servidor** — a API é *stateless*. O fluxo:

1. O usuário envia e-mail e senha para `POST /api/v1/auth/login`.
2. O Spring Security confere a senha contra o hash guardado no banco.
3. Se válida, o `JwtService` **gera um token JWT** assinado e o devolve.
4. A cada requisição seguinte, o frontend envia o token no cabeçalho `Authorization: Bearer <token>`.
5. O `JwtAuthenticationFilter` intercepta a requisição, valida o token e marca o usuário como autenticado **apenas para aquela chamada**.

Características do token (implementação real):

Aspecto	Valor
Algoritmo de assinatura	HMAC-SHA (HS256) — <code>Keys.hmacShaKeyFor(secret)</code>
Conteúdo (claims)	<code>subject</code> = e-mail do usuário, <code>issuedAt</code> , <code>expiration</code>
Validade	24 horas (<code>app.jwt.expiration=86400000</code> ms)
Chave secreta	Variável <code>JWT_SECRET</code> (com valor padrão só para desenvolvimento)
Refresh token	Não há — quando expira, o usuário faz login de novo

O que o token NÃO contém: a senha. Ele guarda apenas o e-mail e os horários. Como é assinado com a chave secreta, ninguém consegue forjá-lo sem conhecê-la — qualquer alteração no conteúdo invalida a assinatura.

Validação do token (no filtro): confere a **assinatura** (íntegra e gerada pela nossa chave) e a **expiração** (não vencido). Se qualquer uma falhar, a requisição segue sem autenticação e cai em `401`.

4.2 Senha: hash BCrypt — tem salt?

Sim, tem salt — e é automático. A senha é guardada com `BCryptPasswordEncoder` (Spring Security). O algoritmo `bcrypt`:

- Gera um **salt aleatório próprio para cada senha** (16 bytes), automaticamente.
- **Embute o salt dentro do próprio hash** — não precisamos guardar o salt em coluna separada.
- Aplica um **fator de custo** (padrão **10**, ou seja, $2^{10} = 1024$ rodadas) que torna a quebra por força bruta lenta.

Formato do que fica salvo na coluna `senha_hash`:

```
$2a$10$N9qo8uL01ckgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy
```

|
| └──┘
| salt (22 chars) + hash resultante (31 chars)
| └─ fator de custo = 10
| └─ identificador do algoritmo (bcrypt)

Consequência prática: dois usuários com a **mesma senha** terão **hashes diferentes**, porque o salt é diferente. Isso impede ataques de *rainbow table* e esconde senhas iguais.

Salt × Pepper: o salt (aleatório, por senha) está presente. **Não** usamos *pepper* (um segredo extra global) nesta versão — é uma melhoria possível de citar como trabalho futuro.

A verificação no login compara a senha digitada com o hash usando `passwordEncoder.matches(...)`, que re-aplica o mesmo salt/custo e confere — **a senha original nunca é desfeita** (hash é via de mão única).

4.3 Autorização (rotas públicas × protegidas)

Configurado em `SecurityConfig`:

- **Públicas** (sem token): `/api/v1/auth/**` (login e registro), `/api/health`, `/api/v1/health`, Swagger e `/error`.
- **Protegidas** (exigem token válido): **todo o resto**.

Além disso, **cada usuário só enxerga os próprios dados**. Os serviços sempre filtram pelo usuário logado — ex.: `transacaoRepository.findByIdAndUsuarioId(id, usuarioId)`. Mesmo que alguém adivinhe o `id` de uma transação de outra pessoa, a consulta não retorna nada (isolamento por usuário).

4.4 CORS

A API libera o frontend (que roda em outra porta) via `CorsConfigurationSource`, permitindo os métodos GET/POST/PUT/PATCH/DELETE. Em produção, o ideal é restringir a origem ao domínio real do frontend.

4.5 Onde o token fica no frontend

O token é guardado no **localStorage** do navegador (chave `glass-finance-auth`, persistido pelo Zustand). O *interceptor* do Axios lê esse token e o injeta no cabeçalho de toda requisição. Se a API responder `401`, o frontend limpa o storage e redireciona para o login.

Ponto de honestidade para a banca: guardar JWT em `localStorage` é simples e comum, mas fica exposto a ataques XSS. Alternativa mais segura: cookie `httpOnly`. Citar isso mostra domínio do assunto.

5. Frontend (organização)

SPA em React com TypeScript. Principais pastas em `frontend/src`:

Pasta	Conteúdo
<code>pages/</code>	Telas: Login, Registro, Painel (dashboard), Transações, Categorias, Perfil.
<code>services/</code>	Funções que chamam a API (um arquivo por recurso).
<code>store/</code>	Estado global com Zustand (<code>autenticacao.ts</code> guarda usuário e token).
<code>components/common/</code>	Componentes reutilizáveis (cartões, gráficos, diálogos, rota privada).
<code>types/</code>	Interfaces TypeScript espelhando os DTOs da API.
<code>assets/styles/</code>	Tema "glass" (vidro), variáveis CSS e animações.

Proteção de rotas: o componente `RotaPrivada` impede o acesso às telas internas sem autenticação. Ao abrir o app, o `checkAuth()` valida o token chamando `/usuarios/perfil`; se falhar, faz logout.

Resiliência (cold start do Render): o Axios tem `timeout` de 90 s e mensagens amigáveis, porque no plano gratuito do Render o backend pode estar "dormindo" e leva alguns segundos para acordar.

6. Integrações com APIs externas

O sistema não consome nenhuma API de terceiros. Não há gateway de pagamento, Open Finance, envio de e-mail ou serviço externo. Toda a comunicação é **frontend ↔ backend próprio**.

O único elemento "externo" é a **hospedagem no Render** (plano gratuito), que pode suspender o serviço após inatividade. Para isso existe:

- O endpoint público e leve `GET /api/health`, usado para verificar/acordar a API.
- A job `MonitoramentoJobs`, que registra um *heartbeat* no log a cada 2 minutos enquanto a aplicação está ativa.

Se a banca perguntar sobre "APIs externas", a resposta correta é: **não há integrações externas**; a API consumida pelo frontend é a do próprio projeto. Esse é um recorte proposital da versão simplificada.

7. Regras de negócio que valem destacar

1. **Categorias padrão no cadastro** — ao criar a conta, o sistema já insere categorias (Salário, Alimentação, Transporte...) para o usuário não começar com a tela vazia.
2. **Dashboard é calculado, não armazenado** — não existe tabela de "resumo"; os números são somados a partir das transações no momento da consulta.
3. **Lançamento automático da renda mensal** — todo 5º dia útil, a renda configurada no perfil vira uma receita "Salário". Um registro por mês (`UNIQUE(usuario_id, ano, mes)`) evita duplicidade. A mesma rotina roda ao cadastrar a conta, ao salvar a renda no perfil e por um botão manual, atualizando o dashboard na hora.
4. **Integridade ao excluir** — uma categoria com transações **não** pode ser excluída (evita transações órfãs).

8. Roteiro de defesa — perguntas prováveis da banca

A banca pode pedir alterações ao vivo (prevenção contra uso indevido de IA). Estude estas respostas e saiba **onde** mexer no código.

Pergunta	Resposta curta	Onde está no código
Como funciona o login?	Valida senha com BCrypt e devolve um JWT assinado (HS256, 24h).	<code>AuthService.login</code> , <code>JwtService</code>
A senha é guardada como?	Hash bcrypt com salt aleatório embutido (custo 10); nunca em texto puro.	<code>Usuario.senhaHash</code> , <code>SecurityConfig.passwordEncoder</code>

Pergunta	Resposta curta	Onde está no código
O hash tem salt?	Sim, o bcrypt gera um salt único por senha automaticamente.	<code>BCryptPasswordEncoder</code>
Como o token é validado a cada requisição?	Um filtro lê o <code>Bearer</code> , confere assinatura e expiração e identifica o usuário.	<code>JwtAuthenticationFilter</code>
Um usuário vê dados de outro?	Não; toda consulta filtra pelo usuário logado.	<code>*Service.buscarEntidade</code> , <code>findBy...AndUsuarioId</code>
O que é stateless?	O servidor não guarda sessão; cada requisição se identifica pelo token.	<code>SecurityConfig</code> (<code>SessionCreationPolicy.STATELESS</code>)
Como o dashboard é montado?	Somando as transações do período direto do banco.	<code>DashboardService.gerar</code>
Como criar um novo campo/endpoint?	Entidade → DTO → Repository → Service → Controller.	seguir o fluxo das camadas
Por que UUID e não id sequencial?	Difícil de adivinhar (não dá para enumerar registros) e bom para sistemas distribuídos.	<code>BaseEntity</code>

Dica de apresentação: mostre o sistema rodando (login → lançar uma transação → ver o dashboard mudar → categorias → perfil). Use dados fictícios rápidos. Deixe ~70% do tempo para a demonstração e o resto para explicar arquitetura/segurança.

Segurança — Documento Técnico Detalhado

Documento aprofundado da camada de segurança do **Controle Financeiro**, preparado para defesa diante de banca especializada. Cada mecanismo é descrito com o **arquivo**, o **método**, o **algoritmo** e o **porquê** da decisão.

Todos os caminhos abaixo partem de `backend/src/main/java/com/financas`.

1. Panorama: o que protege o quê

Camada	Componente	Protege contra
Armazenamento de senha	<code>BCryptPasswordEncoder</code>	Vazamento de senhas se o banco for comprometido.
Autenticação	<code>AuthenticationManager</code> + <code>UsuarioDetailsService</code>	Login com credenciais inválidas.
Emissão de identidade	<code>JwtService.gerarToken</code>	Necessidade de re-enviar senha a cada requisição.
Verificação por requisição	<code>JwtAuthenticationFilter</code>	Acesso sem token / com token forjado ou expirado.
Autorização	<code>SecurityConfig</code> (filter chain)	Acesso a endpoints protegidos sem autenticação.
Isolamento de dados	Consultas <code>...AndUsuarioId(...)</code>	Um usuário acessar dados de outro (IDOR).
Transporte	CORS + (HTTPS em produção)	Origem não autorizada; interceptação.

2. Senha: hash, salt e algoritmo

2.1 Onde o hash é gerado

Arquivo: `config/SecurityConfig.java`

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Esse bean é injetado em quem precisa lidar com senha:

- `AuthService.registrar` → `passwordEncoder.encode(dados.senha())` (linha 49) — ao criar a conta.
- `UsuarioService.alterarSenha` → `passwordEncoder.encode(dados.novaSenha())` (linha 61) — ao trocar a senha.
- `UsuarioService.alterarSenha` → `passwordEncoder.matches(...)` (linha 57) — confere a senha atual.

2.2 Algoritmo: bcrypt

A senha **nunca** é guardada em texto puro. Usamos **bcrypt**, um algoritmo de hash projetado especificamente para senhas (família *Blowfish*). Características:

- **Função de mão única (one-way)**: dado o hash, não há como voltar à senha original.
- **Lento de propósito (key stretching)**: aplica um **fator de custo** (padrão **10** → $2^{10} = 1024$ iterações). Isso torna a força bruta cara, mesmo com GPU.
- **Salt embutido e automático**: ver abaixo.

2.3 Onde o salt entra — "o valor aleatório na frente do hash"

Você não chama o salt manualmente — o bcrypt gera e gerencia. A cada `encode(...)`, o bcrypt:

1. Sorteia um **salt aleatório de 16 bytes** (via `SecureRandom`).
2. Combina salt + senha e roda as 1024 iterações.
3. **Grava o salt dentro da própria string do hash**, logo "na frente" do resultado.

Formato armazenado na coluna `usuario.senha_hash`:

```
$2a$10$Q9mZ3kP1nW8vY2cR4tU6e8XyZ...restante
├──┬──┬──────────────────────────┬──────────┬
│  │  │                          │             │
│  │  │                          │             │ hash final (31 chars)
│  │  │                          │             │
│  │  │                          │             │ SALT aleatório (22 chars Base64) ← "o valor da frente"
│  │  │                          │             │
│  │  └── fator de custo = 10
│  └── versão do algoritmo (bcrypt, $2a$)
```

Por isso o hash muda toda vez: se você cadastrar dois usuários com a senha `123456`, os hashes serão **diferentes**, porque o salt é diferente em cada um. Demonstre isso na banca cadastrando dois usuários com a mesma senha e olhando a tabela `usuario`.

Salt × Pepper:

- **Salt (presente)**: aleatório, **único por senha**, guardado junto ao hash. Derrota *rainbow tables* e esconde senhas iguais.

- **Pepper (ausente nesta versão):** seria um segredo **global** somado à senha antes do hash, guardado **fora** do banco (ex.: variável de ambiente). É uma melhoria possível — citar como trabalho futuro.

2.4 Como o sistema confere se a senha está correta

No bcrypt **não se descriptografa**. A verificação reaplica o mesmo processo:

1. `matches(senhaDigitada, hashGuardado)` lê o **salt** e o **custo** de dentro do `hashGuardado`.
2. Reprocessa a `senhaDigitada` com esse mesmo salt/custo.
3. Compara o resultado com o hash guardado. Igual → senha correta.

No **login**, isso acontece dentro do `AuthenticationManager` (Spring Security), que usa o `UsuarioDetailsService` para carregar o hash e o `BCryptPasswordEncoder` para comparar — ver seção 4.

3. Token JWT: o que é, onde nasce, o que carrega

Arquivo: `security/JwtService.java`

3.1 O que é um JWT

Um **JSON Web Token** tem três partes separadas por ponto: `header.payload.signature`.

- **Header:** algoritmo da assinatura (aqui, **HS256** = HMAC-SHA256).
- **Payload (claims):** dados não secretos. No nosso caso: `sub` (e-mail), `iat` (emitido em), `exp` (expira em).
- **Signature:** HMAC-SHA256 de `header.payload` usando a **chave secreta** do servidor.

O payload **não é criptografado**, só assinado. Por isso **não guardamos senha nem dados sensíveis** ali — apenas o e-mail. A assinatura garante **integridade** (ninguém altera o conteúdo sem invalidar o token), não sigilo.

3.2 Onde o token é criado

`JwtService.gerarToken(String email)` — linhas 33-42:

```
public String gerarToken(String email) {
    Date agora = new Date();
    Date expira = new Date(agora.getTime() + expiracaoMs); // +24h
    return Jwts.builder()
        .subject(email)           // sub = e-mail do usuário
```

```

        .issuedAt(agora)           // iat
        .expiration(expira)        // exp
        .signWith(chave)           // assina com HMAC-SHA (HS256)
        .compact();
    }

```

Quem chama: `AuthService.montarResposta` (linha 73), tanto no **registro** quanto no **login**. O token volta no corpo da resposta (`AuthResponse.accessToken`).

3.3 A chave secreta e o algoritmo

Construtor do `JwtService` (linhas 25-30):

```

this.chave = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));

```

- `secret` vem de `app.jwt.secret` (variável `JWT_SECRET`, com fallback só para desenvolvimento).
- `Keys.hmacShaKeyFor(...)` cria uma chave **HMAC-SHA**. Como a chave do nosso secret tem ≥ 256 bits, a biblioteca (JJWT) assina em **HS256**.
- **HMAC** = *Hash-based Message Authentication Code*: combina a mensagem com a chave secreta e aplica SHA-256. Só quem tem a chave consegue gerar/validar a assinatura.

3.4 Validade

`app.jwt.expiration=86400000` ms = **24 horas**. Não há *refresh token*: ao expirar, o usuário faz login de novo. (Trade-off consciente: simplicidade × conveniência.)

4. Fluxo de LOGIN (passo a passo, com algoritmos)

Arquivos: `controller/AuthController.java` → `service/AuthService.java`

```

POST /api/v1/auth/login  { email, senha }
    |
    v
AuthController.login(dados)                                AuthController.java
    |
    v
AuthService.login(dados)                                    AuthService.java:61
    |
    └─ authenticationManager.authenticate(                  linha 63
        new UsernamePasswordAuthenticationToken(email, senha))
        |
        └─ chama UsuarioDetailsService.loadUserByUsername(email)
            └─ → carrega Usuario e devolve User(email, senhaHash, [])
                UsuarioDetailsService.java:25

```

```

    DaoAuthenticationProvider usa BCryptPasswordEncoder.matches(
        senhaDigitada, senhaHash)
        • senha errada → lança BadCredentialsException

    → (se passou) usuarioRepository.findByEmail(email)          linha 66
    → montarResposta(usuario) → jwtService.gerarToken(email)    linha 73
    ↓
    AuthResponse { accessToken, tokenType="Bearer", expiresIn, usuario }

```

Senha errada: o `AuthenticationManager` lança `BadCredentialsException`, capturada pelo `GlobalExceptionHandler` (linha 29-31), que devolve **HTTP 401** com a mensagem genérica "E-mail ou senha inválidos" (mensagem genérica de propósito, para não revelar se o e-mail existe).

5. Fluxo de toda REQUISIÇÃO autenticada (o "filter chain")

Arquivos: `config/SecurityConfig.java` + `security/JwtAuthenticationFilter.java`

5.1 Configuração do filtro (quem é público × protegido)

`SecurityConfig.securityFilterChain` define:

- **Stateless:** `SessionCreationPolicy.STATELESS` — o servidor **não guarda sessão**; cada requisição se identifica sozinha pelo token.
- **Públicos:** `/api/v1/auth/**`, `/api/health`, `/api/v1/health`, Swagger, `/error`.
- **Protegidos:** `anyRequest().authenticated()` — todo o resto exige autenticação.
- O `JwtAuthenticationFilter` é inserido **antes** do `UsernamePasswordAuthenticationFilter`:

```

.addFilterBefore(jwtAuthenticationFilter,
UsernamePasswordAuthenticationFilter.class)

```

- **Resposta a não autenticado = 401:** configuramos um `authenticationEntryPoint` que devolve **401 Unauthorized** quando falta token (em vez do **403** padrão do Spring). Assim distinguimos *não autenticado* (401) de *autenticado sem permissão* (403), como manda o HTTP — e isso é verificado pelo teste de integração (ver `docs/TESTES-AUTOMATIZADOS.md`).

5.2 O filtro, linha a linha — `JwtAuthenticationFilter.doFilterInternal`

```

String header = request.getHeader("Authorization");          // linha 37
if (header != null && header.startsWith("Bearer ")) {        // linha 39

```

```

        String token = header.substring(7); // linha 40 (tira
"Bearer ")
        if (jwtService.tokenValido(token) // linha 42 (assinatura
+ expiração)
            && SecurityContextHolder.getContext().getAuthentication() == null) {
            String email = jwtService.extrairEmail(token); // linha 45 (lê o
"sub")
            UserDetails usuario =
                usuarioDetailsService.loadUserByUsername(email); // linha 46 (carrega do
banco)
            var auth = new UsernamePasswordAuthenticationToken(
                usuario, null, usuario.getAuthorities()); // linha 48-49
            (authorities = papéis)
            auth.setDetails(...); // linha 50
            SecurityContextHolder.getContext()
                .setAuthentication(auth); // linha 51 (marca como
autenticado)
        }
    }
    filterChain.doFilter(request, response); // linha 55 (segue o
fluxo)

```

5.3 Como o token é validado — JwtService.tokenValido / lerClaims

```

public boolean tokenValido(String token) { // linha 50
    try {
        return lerClaims(token).getExpiration().after(new Date()); // não expirou?
    } catch (Exception e) {
        return false; // assinatura
        inválida/malformado → false
    }
}

private Claims lerClaims(String token) { // linha 58
    return Jwts.parser()
        .verifyWith(chave) // confere a assinatura HMAC com a NOSSA chave
        .build()
        .parseSignedClaims(token) // lança exceção se a assinatura não bater
        .getPayload();
}

```

Ou seja, a validação cobre **duas coisas**: (1) **assinatura** — o token foi assinado pela nossa chave e não foi adulterado; (2) **expiração** — ainda está dentro das 24h. Qualquer falha → o filtro **não autentica** e a requisição cai em **401** ao tentar acessar rota protegida.

5.4 Tipo de permissão do usuário (authorities)

Hoje, `UsuarioDetailsService.loadUserByUsername` cria o usuário com **lista de papéis vazia** (`List.of()`, linha 30). Ou seja, **não há perfis diferenciados** (admin/comum) nesta versão: a

autorização é binária — *autenticado* × *não autenticado* — e o isolamento por usuário (seção 6) faz o resto.

Pergunta provável da banca: *"E se quisesse um admin?"* — Resposta: bastaria popular as authorities (ex.: `List.of(new SimpleGrantedAuthority("ROLE_ADMIN"))`) e proteger endpoints com `@PreAuthorize("hasRole('ADMIN')")`. A estrutura já suporta; só não foi necessário no escopo.

6. Autorização de dados — isolamento por usuário (anti-IDOR)

Mesmo autenticado, o usuário só acessa o que é dele. Isso é garantido **na consulta**, não só na URL:

- `UsuarioService.getLogado()` (linha 35-38) lê o e-mail do `SecurityContext` e busca o `Usuario` atual.
- As consultas sempre amarram o `usuarioId`:
 - `transacaoRepository.findByIdAndUsuarioId(id, usuarioId)`
 - `categoriaRepository.findByIdAndUsuarioId(id, usuarioId)`

Efeito: se o usuário A tentar `GET /api/v1/transacoes/{id}` com o `id` de uma transação do usuário B, a consulta retorna vazio → **404**, não os dados de B. Isso previne **IDOR** (*Insecure Direct Object Reference*), uma falha clássica que a banca pode testar.

7. Outras decisões e pontos de honestidade

Tema	Situação atual	Observação para a banca
Transporte	HTTP em dev	Em produção, HTTPS/TLS obrigatório (o token viaja no header).
Armazenamento do token no front	<code>localStorage</code>	Simples, porém exposto a XSS . Alternativa: cookie <code>httpOnly</code> .
CORS	<code>allowedOriginPatterns("*")</code>	Em produção, restringir ao domínio real do frontend.
Segredo JWT	Variável <code>JWT_SECRET</code>	O fallback do código é só para dev; em produção, segredo forte e fora do código.
Revogação de token	Não há lista de revogação	Como é stateless, um token válido vale até expirar. Mitigação: expiração curta / blacklist (trabalho futuro).

Tema	Situação atual	Observação para a banca
Força da senha	Validação básica no DTO	Pode-se reforçar política de senha (tamanho, complexidade).
Proteção de força bruta no login	Não há rate limit	Citar como melhoria (ex.: bloqueio após N tentativas).

Demonstrar consciência desses limites **conta a favor** na defesa — mostra que você entende segurança, não só implementou.

8. Resumo de algoritmos e bibliotecas

Função	Algoritmo / Tecnologia	Biblioteca
Hash de senha	bcrypt (custo 10, salt aleatório)	Spring Security Crypto
Assinatura do token	HMAC-SHA256 (HS256)	JWT (io.jsonwebtoken 0.12.5)
Geração de salt	SecureRandom (dentro do bcrypt)	Spring Security Crypto
Identidade/sessão	JWT stateless	JWT + Spring Security
Controle de acesso	Filter chain + SecurityContext	Spring Security 6

Guia de Breakpoints — Demonstração ao Vivo

Roteiro para depurar (*debug*) o backend na frente da banca e mostrar o sistema funcionando "por dentro". Para cada cenário há **onde** colocar o breakpoint, **o que** inspecionar e **o que dizer**.

Caminhos a partir de `backend/src/main/java/com/financas`.

Como iniciar o modo debug

IntelliJ IDEA / VS Code:

1. Abra a classe `FinancasApplication`.
2. Rode em modo **Debug** (ícone do inseto / `Run > Debug`), em vez de Run normal.
3. Coloque os breakpoints clicando na margem esquerda (na linha indicada).
4. Faça a ação no frontend (ou no Postman) e a execução vai **parar** no breakpoint.
5. Use **Step Over (F8)** para avançar linha a linha e **Resume (F9)** para continuar.

Dica: mantenha aberta a aba *Variables* (ou *Debugger*) para mostrar os valores em tempo real (e-mail, token, hash da senha etc.).

Cenário 0 — A "espinha dorsal": toda requisição passa pelo filtro

Este é o breakpoint mais importante para o professor de segurança. Mostra que **toda** chamada protegida passa pela validação do token **antes** de chegar ao controller.

Breakpoint: `security/JwtAuthenticationFilter.java`

Linha	O que mostrar
37	<code>request.getHeader("Authorization")</code> — o token chegando no header.
42	<code>jwtService.tokenValido(token)</code> — entre aqui (Step Into) para mostrar a checagem de assinatura + expiração.
45	<code>jwtService.extrairEmail(token)</code> — o e-mail é lido de dentro do token.
46	<code>loadUserByUsername(email)</code> — o usuário é carregado do banco.
51	<code>setAuthentication(auth)</code> — a partir daqui o usuário está autenticado para esta requisição.

O que dizer: "Cada requisição a uma rota protegida para aqui primeiro. Se o token for inválido ou expirado, `tokenValido` devolve false, não autenticamos, e o Spring Security responde 401.

Repare que o servidor não guarda sessão — a identidade vem inteira do token."

Demonstração extra: edite o token no Postman (troque um caractere) e refaça a requisição → mostre `tokenValido` retornando **false**.

Cenário 1 — Criar uma conta nova (registro)

Fluxo: `AuthController.registrar` → `AuthService.registrar`

Breakpoints: `service/AuthService.java`

Linha	O que mostrar
42	<code>existsByEmail(...)</code> — verificação de e-mail duplicado.
49	<code>passwordEncoder.encode(dados.senha())</code> — a senha virando hash bcrypt . Inspecione o retorno: começa com <code>\$2a\$10\$...</code> . Compare com <code>dados.senha()</code> (texto puro que entrou).
52	<code>save(usuario)</code> — o usuário é persistido (com o hash, nunca a senha pura).
54	<code>criarCategoriasPadrao(usuario)</code> — as 6 categorias iniciais.
55	<code>executarParaUsuario(...)</code> — já lança a renda do mês, se houver.
57	<code>montarResposta(usuario)</code> — entra no <code>gerarToken</code> (Cenário 4).

O que dizer: "No cadastro, a senha digitada é transformada em hash bcrypt na linha 49. O que vai para o banco é o hash com salt embutido — a senha original é descartada. Em seguida já criamos categorias padrão e devolvemos um token, então o usuário entra logado."

Demonstração de salt: cadastre **dois** usuários com a mesma senha e mostre, parando na linha 49, que os hashes gerados são **diferentes**.

Cenário 2 — Cadastrar um gasto (transação)

Fluxo: filtro JWT (Cenário 0) → `TransacaoController.criar` → `TransacaoService.criar`

Breakpoints: `service/TransacaoService.java`

Linha	O que mostrar
45	<code>usuarioService.getLogado()</code> — entre aqui para mostrar que o usuário vem do <code>SecurityContext</code> (preenchido pelo filtro), não do corpo da requisição.
46	<code>buscarCategoria(dados.categoriaId(), usuario.getId())</code> — isolamento: a categoria é buscada amarrada ao <code>usuarioId</code> . Se não for dele, dá 404.

Linha	O que mostrar
48-55	Montagem da entidade <code>Transacao</code> (tipo, valor, descrição, data).
57	<code>transacaoRepository.save(transacao)</code> — grava no banco e devolve o DTO.

O que dizer: "Quem é o dono do gasto não vem do front — vem do token, via `getLogado()`. E a categoria é validada contra o usuário logado, então ninguém lança gasto em categoria de outra pessoa."

Cenário 3 — Criar uma categoria

Fluxo: filtro JWT → `CategoriaController.criar` → `CategoriaService.criar`

Breakpoints: `service/CategoriaService.java`

Linha	O que mostrar
44	<code>usuarioService.getLogado()</code> — o dono da categoria.
45	<code>existsByNomeAndUsuarioId(...)</code> — regra: não pode repetir nome para aquele usuário.
49-57	Montagem da <code>Categoria</code> (nome, ícone, cor, tipo, orçamento).
59	<code>categoriaRepository.save(categoria)</code> — persiste.

O que dizer: "A unicidade do nome é por usuário: dois usuários diferentes podem ter, cada um, a categoria 'Alimentação'. A verificação na linha 45 usa o id do usuário logado."

Cenário 4 — Onde o token é criado (após login ou registro)

Breakpoints: `security/JwtService.java`

Linha	O que mostrar
34-35	<code>agora</code> e <code>expira</code> — a janela de validade (24h).
36-41	A construção do JWT: <code>subject(email)</code> , <code>expiration</code> , <code>signWith(chave)</code> .
40	<code>signWith(chave)</code> — a assinatura HMAC-SHA. Explique que sem a chave secreta ninguém forja um token válido.

Para o login, coloque também: `service/AuthService.java:63` (`authenticationManager.authenticate(...)`). Faça **Step Into** para mostrar que é aí que o Spring compara a senha digitada com o hash (bcrypt `matches`).

O que dizer: "O token nasce aqui, no `gerarToken`. Ele carrega só o e-mail e a expiração, e é assinado com nossa chave secreta na linha 40. O conteúdo é público, mas a assinatura impede adulteração."

Cenário 5 — Alterar a renda mensal no perfil

Fluxo: filtro JWT → `UsuarioController.atualizarPerfil` → `UsuarioService.atualizarPerfil`

Breakpoints: `service/UsuarioService.java`

Linha	O que mostrar
44	<code>getLogado()</code> — pega o usuário do token.
45-46	<code>setNome(...)</code> e <code>setRendaMensal(...)</code> — os novos valores chegando.
47	<code>save(usuario)</code> — persiste a nova renda.
48	<code>rendaMensalJobs.executarParaUsuario(usuario, hoje)</code> — entre aqui (Step Into) para mostrar o efeito colateral: a renda do mês é (re)lançada/atualizada como receita, e o dashboard muda na hora.

O que dizer: "Quando salvo a renda, além de gravar no perfil, disparo a rotina de renda mensal na linha 48. Ela atualiza a receita 'Salário' do mês corrente — por isso o dashboard reflete a mudança imediatamente, sem esperar o dia 5."

Aprofundar (opcional): breakpoint em `jobs/RendaMensalJobs.executarParaUsuario` e em `processarUsuario` para mostrar a checagem de duplicidade (um lançamento por usuário/mês).

Bônus — Trocar a senha (mostra o matches)

Breakpoint: `service/UsuarioService.java:57`

```
if (!passwordEncoder.matches(dados.senhaAtual(), usuario.getSenhaHash())) {
```

Mostre que, para trocar a senha, o sistema **confere a senha atual** com `matches` (bcrypt reaplica o salt do hash guardado). Senha atual errada → exceção e **400**, sem trocar nada. Na linha 61, a nova senha vira um **novo hash** (com salt novo).

Tabela-resumo dos breakpoints

Cenário	Arquivo	Linha(s)	Conceito demonstrado
Toda requisição	JwtAuthenticationFilter	37, 42, 45, 46, 51	Validação de token / autenticação stateless
Registro	AuthService	42, 49, 52, 57	Hash da senha + criação da conta
Gasto	TransacaoService	45, 46, 57	Isolamento por usuário
Categoria	CategoriaService	44, 45, 59	Unicidade por usuário
Criação do token	JwtService	34, 40	Assinatura HMAC / expiração
Login (senha)	AuthService	63	Comparação bcrypt (matches)
Renda mensal	UsuarioService	44, 47, 48	Efeito colateral / job
Troca de senha	UsuarioService	57, 61	matches + novo hash

Guia de Testes da API (Postman / curl)

Como testar a API manualmente na apresentação, mostrando autenticação, CRUD e o bloqueio de quem não tem token.

Importar a coleção no Postman

1. Abra o Postman → **Import**.
2. Selecione `docs/postman/ControleFinanceiro.postman_collection.json`.
3. A coleção já vem com a variável `baseUrl = http://localhost:8080`.
4. Rode na ordem numerada. Os scripts de teste **salvam o token automaticamente** na variável `{{token}}` após o registro/login — as demais requisições já o usam.

Ordem recomendada (roteiro de demonstração)

#	Requisição	Espera-se	Mostra
0	GET <code>/api/health</code>	200	API no ar (rota pública).
1	POST <code>/api/v1/auth/registro</code>	201 + token	Criação de conta; token salvo.
2	POST <code>/api/v1/auth/login</code>	200 + token	Autenticação; token salvo.
3	GET <code>/api/v1/usuarios/perfil</code>	200	Token funcionando (rota protegida).
4	GET <code>/api/v1/categorias</code>	200 + lista	Categorias padrão criadas no registro.
5	POST <code>/api/v1/categorias</code>	201	Criar categoria.
6	POST <code>/api/v1/transacoes</code>	201	Lançar um gasto.
7	GET <code>/api/v1/transacoes?page=0&size=20</code>	200	Listagem paginada.
8	GET <code>/api/v1/dashboard?periodo=2026-06</code>	200	Resumo financeiro calculado.
9	PUT <code>/api/v1/usuarios/perfil</code>	200	Alterar renda mensal.
10	GET <code>/api/v1/transacoes</code> sem token	401	Prova de que a rota é protegida.

A requisição **10** é a mais importante para o professor de segurança: ela mostra que, sem o cabeçalho `Authorization`, o `JwtAuthenticationFilter` não autentica e o Spring Security

responde 401.

Tabela completa de endpoints

Método	Rota	Auth	Corpo (exemplo)
GET	/api/health	não	—
POST	/api/v1/auth/registro	não	{ nome, email, senha, rendaMensal? }
POST	/api/v1/auth/login	não	{ email, senha }
GET	/api/v1/usuarios/perfil	sim	—
PUT	/api/v1/usuarios/perfil	sim	{ nome, rendaMensal }
PUT	/api/v1/usuarios/senha	sim	{ senhaAtual, novaSenha }
GET	/api/v1/categorias?tipo=	sim	—
POST	/api/v1/categorias	sim	{ nome, icone, cor, tipo, orcamento?, descricao? }
PUT	/api/v1/categorias/{id}	sim	igual ao POST
DELETE	/api/v1/categorias/{id}	sim	—
GET	/api/v1/transacoes? page=&size=	sim	—
POST	/api/v1/transacoes	sim	{ tipo, valor, descricao, dataTransacao, categoriaId }
PUT	/api/v1/transacoes/{id}	sim	igual ao POST
DELETE	/api/v1/transacoes/{id}	sim	—
GET	/api/v1/dashboard? periodo=yyyy-MM	sim	—

Testando por linha de comando (curl)

Caso prefira o terminal:

```
# 1) Login e captura do token
TOKEN=$(curl -s -X POST http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"maria@teste.com","senha":"123456"}' | sed -n 's/.*"accessToken":"(\\[^\"]*\\)".*/\\1/p')
```



```
echo "Token: $TOKEN"

# 2) Rota protegida usando o token
curl -s http://localhost:8080/api/v1/usuarios/perfil \
  -H "Authorization: Bearer $TOKEN"

# 3) Criar um gasto (troque o categoriaId por um id real vindo de /categorias)
curl -s -X POST http://localhost:8080/api/v1/transacoes \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"tipo":"DESPESA","valor":89.90,"descricao":"Mercado","dataTransacao":"2026-06-12","categoriaId":"COLE_AQUI"}'

# 4) Prova de bloqueio: sem token deve responder 401
curl -s -o /dev/null -w "HTTP %{http_code}\n" http://localhost:8080/api/v1/transacoes
```

Alternativa: Swagger UI

A API também expõe documentação interativa (já testável no navegador) em:

```
http://localhost:8080/swagger-ui.html
```

No Swagger, use o botão **Authorize** e cole `Bearer <token>` para testar as rotas protegidas direto pela tela.

Testes Automatizados (JUnit)

O projeto tem testes automatizados que provam, sem intervenção manual, que a segurança e o fluxo principal funcionam. São de dois tipos: **unitário** e **integração**.

Como rodar

```
cd backend
mvn test
```

Os testes usam um banco **H2** em **memória** (configurado em `backend/src/test/resources/application.properties`), então **não é preciso ter o PostgreSQL rodando**. Ao final, o Maven mostra algo como:

```
Tests run: 8, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

O que cada teste prova

1. Teste UNITÁRIO — JwtServiceTest

Arquivo: `backend/src/test/java/com/financas/security/JwtServiceTest.java`

Testa o `JwtService` isolado (sem subir o Spring, sem banco). Casos:

Teste	O que garante
<code>gerarToken_eExtrairEmail</code>	O e-mail colocado no token é o mesmo que se lê de volta.
<code>tokenRecemCriado_eValido</code>	Token dentro da validade é aceito.
<code>tokenExpirado_eInvalido</code>	Token vencido é rejeitado.
<code>tokenComOutraChave_eInvalido</code>	Token assinado com outra chave é rejeitado (anti-falsificação).
<code>textoQualquer_aoEInvalido</code>	Texto que não é um JWT é tratado como inválido, sem quebrar.

O caso `tokenComOutraChave_eInvalido` é ótimo para a banca de segurança: prova na prática que, sem a chave secreta, **não dá para forjar um token válido**.

2. Teste de INTEGRAÇÃO — AuthFlowIntegrationTest

Arquivo: `backend/src/test/java/com/financas/integracao/AuthFlowIntegrationTest.java`

Sobe a aplicação inteira (com H2) e exercita o caminho real via `MockMvc`. Casos:

Teste	O que garante
<code>fluxoCompletoDeAutenticacao</code>	Registrar (201) → Login (200, devolve token) → acessar <code>/usuarios/perfil</code> com token (200) → sem token (401).
<code>loginComSenhaErrada_eRejeitado</code>	Login com senha errada devolve 401.
<code>registroComEmailDuplicado_eRejeitado</code>	Cadastrar e-mail repetido devolve 409.

Tipos de teste — a diferença (para explicar na banca)

	Unitário	Integração
Escopo	Uma classe isolada (<code>JwtService</code>).	Várias camadas juntas (Controller → Service → Repository → Banco).
Sobe o Spring?	Não.	Sim (<code>@SpringBootTest</code>).
Usa banco?	Não.	Sim (H2 em memória).
Velocidade	Muito rápido.	Mais lento (sobe o contexto).
Prova	A lógica de uma peça.	Que as peças funcionam juntas .

Nota técnica: por que 401 e não 403

Durante a escrita dos testes, identificamos que o Spring Security, por padrão, devolvia **403 (Forbidden)** para requisições **sem token** em rotas protegidas. O correto, segundo o HTTP, é:

- **401 Unauthorized** → não autenticado (sem token / token inválido);
- **403 Forbidden** → autenticado, mas sem permissão.

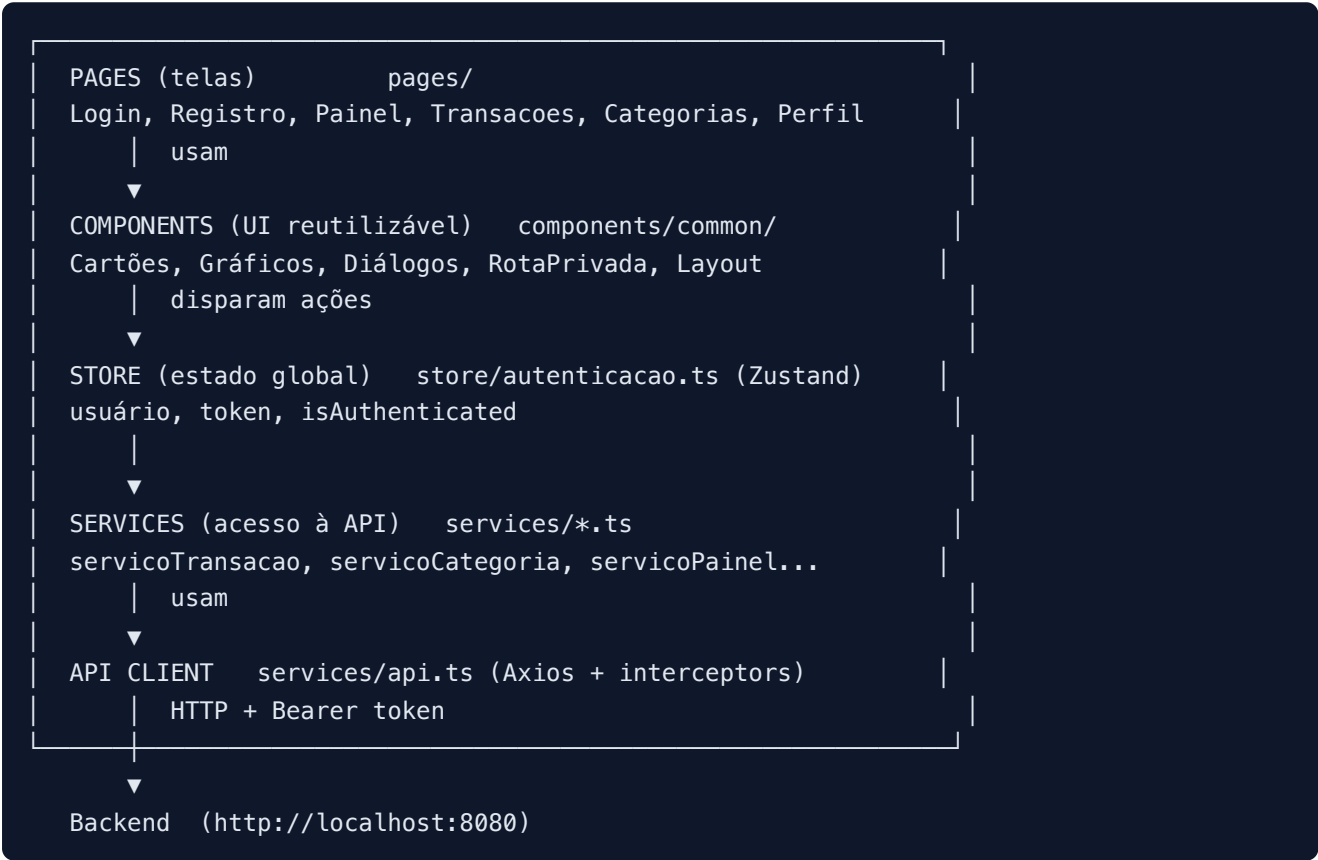
Ajustamos o `SecurityConfig` para devolver **401** quando não há autenticação (via `authenticationEntryPoint`). Isso, além de semanticamente correto, **alinha o backend com o frontend**, cujo interceptor do Axios trata o 401 para redirecionar ao login quando o token expira. O teste `fluxoCompletoDeAutenticacao` trava esse comportamento (se alguém quebrar, o teste falha).

Arquitetura do Frontend

Como a interface web é organizada, em camadas, e como cada parte se conecta com a API. Caminhos a partir de `frontend/src`.

1. Visão em camadas

O frontend é uma **SPA** (Single Page Application): uma única página HTML que troca de tela sem recarregar, conversando com a API por JSON.



Camada	Pasta	Responsabilidade
Pages	<code>pages/</code>	As telas do sistema (uma por rota).
Components	<code>components/common/</code>	Peças de UI reaproveitadas (cartões, gráficos, diálogos, layout, rota privada).
Store	<code>store/</code>	Estado global com Zustand — guarda usuário e token, persistidos no <code>localStorage</code> .
Services	<code>services/</code>	Funções que chamam a API (uma por recurso). Isolam o componente da URL/HTTP.
API client	<code>services/api.ts</code>	Instância única do Axios com <i>interceptors</i> que anexam o token e tratam erros/401.

Camada	Pasta	Responsabilidade
Types	<code>types/</code>	Interfaces TypeScript que espelham os DTOs da API (tipagem ponta a ponta).
Hooks	<code>hooks/</code>	Lógica reutilizável (ex.: notificação/toast).
Styles	<code>assets/styles/</code>	Tema visual "glass" (vidro), variáveis e animações CSS.

Por que separar `services` das `pages`? Para que a tela não saiba *como* a API é chamada — ela só pede `transacaoService.create(...)`. Se a URL ou o formato mudar, muda-se em um lugar só. É o mesmo princípio do backend (Controller × Service).

2. Roteamento e proteção de telas

Arquivo: `App.tsx`

- Usa **React Router**. Rotas públicas: `/login` e `/registro`. As demais ficam **dentro** de `<PrivateRoute>`.
- `RotaPrivada.tsx` (`components/common`) decide:
 - ainda verificando token → mostra tela de carregamento;
 - não autenticado → redireciona para `/login`;
 - autenticado → libera a tela (`<Outlet/>`).
- **Code splitting**: cada página é carregada sob demanda (`lazy(() => import(...))`), deixando o carregamento inicial mais leve.
- Ao abrir o app, um `useEffect` chama `checkAuth()`: se houver token salvo, confirma no backend (`GET /usuarios/perfil`) se ainda é válido.

3. Autenticação no front (o ciclo do token)

Arquivo: `store/autenticacao.ts` (Zustand com `persist`)

1. **Login/registro**: `authStore.login()` chama `POST /api/v1/auth/login`, recebe `accessToken` e guarda no estado.
2. **Persistência**: o middleware `persist` salva `token` e `user` no `localStorage` (chave `glass-finance-auth`). Assim, ao recarregar a página, o usuário continua logado.
3. **Envio automático**: o *interceptor de request* do Axios (`services/api.ts`) lê o token do `localStorage` e injeta `Authorization: Bearer <token>` em **toda** requisição.
4. **Expiração/erro**: o *interceptor de response* detecta **401**, limpa o storage e redireciona para `/login`.
5. **Logout**: remove o token do estado e do header do Axios.

Login → token no Zustand → persistido no localStorage

```
cada requisição → interceptor Axios lê o token → header Bearer
|
resposta 401 → limpa storage → volta ao /login
```

4. Fluxo completo de uma ação (ex.: lançar um gasto)

```
Tela Transacoes (pages/Transacoes.tsx)
|  usuário preenche o formulário (React Hook Form + Zod valida)
▼
transacaoService.create(dados)          (services/servicoTransacao.ts)
|
▼
api.post('/api/v1/transacoes', dados) (services/api.ts)
|  interceptor anexa Authorization: Bearer <token>
▼
Backend valida o token e grava → devolve a transação criada
|
▼
Tela atualiza a lista e mostra um toast de sucesso
```

5. Bibliotecas principais e o papel de cada uma

Biblioteca	Papel
React 19 + TypeScript	Base da SPA, com tipagem estática.
Vite	Servidor de desenvolvimento e <i>build</i> de produção (rápido).
React Router	Navegação entre telas sem recarregar a página.
Zustand	Estado global simples (usuário/token), com persistência.
Axios	Cliente HTTP com <i>interceptors</i> (token e tratamento de 401).
React Hook Form + Zod	Formulários e validação declarativa dos campos.
PrimeReact + PrimeFlex	Componentes de UI (tabelas, inputs, diálogos) e layout.
Chart.js	Gráficos do dashboard (pizza de gastos, barras de fluxo).
lucide-react / primeicons	Ícones.

6. Resiliência a *cold start* (deploy gratuito)

O Axios usa `timeout: 90000` (90s) e mensagens amigáveis quando o backend está "acordando" no Render gratuito. Se o servidor demora, o usuário vê *"O servidor pode estar iniciando..."* em vez de um erro seco.

7. Pontos para a banca

- **Onde fica o token?** No `localStorage` (via Zustand `persist`). É simples; a alternativa mais segura contra XSS seria cookie `httpOnly`.
- **Quem protege as telas?** O componente `RotaPrivada` no front (UX) e o backend (segurança real). O front nunca é a única defesa — o backend valida o token em toda requisição.
- **Como o token chega na API?** Sempre pelo *interceptor* do Axios, nunca manualmente em cada chamada.